RECEP TAYYIP ERDOGAN UNIVERSITY

FACULTY OF ENGINEERING AND ARCHITECTURE

COMPUTER ENGINEERING DEPARTMENT

DATA MINING

2024-2025

Student Name Surname

Doğukan Erzurum

Student No

201401023

Instructor

Dr. Öğr. Üyesi ABDULGANİ KAHRAMAN

RIZE

1.1. Introduction

This is the report of the 3 question project assignment of the Data mining course. The report continues according to the number of each question. While the titles that continue with 1.x.x are the titles of the 1st question. Headings with 2.x.x belong to question 2. The headings followed by 3.x.x belong to question 3. Do not forget to take this into consideration when reviewing the report.

1.1.1. Project Objective

This project aims to predict a target variable using machine learning algorithms. In particular, the decision tree algorithm will be applied to predict the target variable and the model performance will be evaluated. Furthermore, the performance of the decision tree model will be analyzed by comparing it with an alternative model, Random Forest.Proje sırasında:

- Steps such as data processing, feature selection, and data visualization have been thoroughly addressed.
- Performance evaluation metrics such as Accuracy, Precision, Recall, and F1-Score have been utilized.
- Visualizations like the Confusion Matrix have been used to analyze the strengths and weaknesses of the models.

1.1.2. About the Dataset

In this project, the Diabetes Dataset obtained from the Kaggle platform was used. The dataset includes various health measurements of individuals and indicates whether they have diabetes or not.

- **Total Number of Observations:** 768
- **Number of Columns**: 9
- **Target Variable (Outcome)**:
    - 0: The individual is not diabetic.
    - 1: The individual is diabetic.
- **Independent Variables**:
    - **Pregnancies**: Number of pregnancies.
    - **Glucose**: Blood glucose level.
    - **BloodPressure**: Blood pressure.
    - **SkinThickness**: Skin thickness.
    - **Insulin**: Insulin level.
    - **BMI**: Body Mass Index.
    - **DiabetesPedigreeFunction**: Diabetes risk factor (genetic predisposition).
    - **Age**: Age.

1.1.3. Project Process

The project involves the following steps:

1. **Data Exploration and Preprocessing:**
    - Identifying and handling missing values in the dataset.
    - Limiting outliers.

- Scaling continuous variables.

2. **Modeling**:
   - Training and testing Decision Tree and Random Forest algorithms.
3. **Performance Evaluation:**
   - Using metrics such as Accuracy, Precision, Recall, and F1-Score,
   - Analyzing results through a Confusion Matrix.
4. **Model Comparison and Interpretation:**

   - Comparing the models and interpreting the results.

1.2. Data Exploration and Processing

In this section, we will examine the dataset in detail, handle missing and outlier values, perform feature selection, and apply data preprocessing steps. Our goal is to prepare the dataset for modeling and take the necessary steps to enhance model performance..

1.2.1. Loading and Initial Examination of the Dataset

First, we load the dataset using the pandas library and review its basic information. This includes:.

```
# Gerekli kütüphanelerin import edilmesi
import pandas as pd

# Veri setinin yüklenmesi
data = pd.read_csv('diabetes.csv')

# Veri setinin ilk beş satırının görüntülenmesi
print("Veri Setinin İlk 5 Satırı:")
print(data.head())

# Veri setinin genel bilgileri
print("\nVeri Seti Genel Bilgileri:")
print(data.info())
```

Output:

```
Veri Setinin İlk 5 Satırı:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI
\
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
```

```
2                       0.672   32        1
3                       0.167   21        0
4                       2.288   33        1

Veri Seti Genel Bilgileri:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

Description:

- The dataset contains a total of 768 observations and 9 columns.
- The Outcome column is defined as the target variable, indicating the diabetes status of individuals. This variable is categorized as:
    - 0: Not diabetic,
    - 1: Diabetic,
- Initial examination revealed no missing values in the dataset. However, it was noted that 0 values in some columns might represent missing data. This issue has been identified as an element requiring further analysis.

1.2.2. Analysis of Missing Values

In the dataset, 0 values in certain columns have been identified as biologically implausible measurements. This indicates the presence of missing data in these columns. Analyzing and handling missing values using appropriate methods has been identified as a critical step to enhance the model's accuracy and reliability.

```
# Eksik değerlerin kontrolü
print("\nEksik Değerler Kontrolü:")
print(data.isnull().sum())

# İstatistiksel özet
```

```
print("\nİstatistiksel Özellikler:")
print(data.describe())
```

Çıktı:

```
Eksik Değerler Kontrolü:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
İstatistiksel Özellikler:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

             BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.00000                768.000000  768.000000  768.000000
mean    31.99258                  0.471876   33.240885    0.348958
std      7.88416                  0.331329   11.760232    0.476951
min      0.00000                  0.078000   21.000000    0.000000
25%     27.30000                  0.243750   24.000000    0.000000
50%     32.00000                  0.372500   29.000000    0.000000
75%     36.60000                  0.626250   41.000000    1.000000
max     67.10000                  2.420000   81.000000    1.000000
```

Description:

- According to the missing value check results, no directly missing values were identified in the dataset. However, upon examining the statistical summary, it was observed that the **min** and **25%** values for certain columns are **0**. Specifically, **Glucose, BloodPressure, SkinThickness, Insulin**, and **BMI** columns contain 0 values, which are biologically implausible and can be considered as missing data.
- This indicates the necessity of handling missing data with appropriate methods, which will have a critical impact on the accuracy of the modeling process. Before imputing the missing values, a detailed analysis of data distributions should be conducted, and suitable imputation techniques should be applied.

1.2.3. Marking and Filling Missing Values

To properly handle missing data, **0** values in the **Glucose**, **BloodPressure**, **SkinThickness**, **Insulin**, and **BMI** columns were treated as missing and replaced with **NaN**. After this operation, the number of missing values was determined as follows:

```
# Eksik veri olarak değerlendirilen 0'ları NaN ile işaretleme
import numpy as np

columns_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness',
'Insulin', 'BMI']
data[columns_with_zeros] = data[columns_with_zeros].replace(0,
np.nan)

# Eksik değerlerin sayısının kontrolü
print("\nEksik Değerler (0'lar NaN olarak işaretlendikten sonra):")
print(data.isnull().sum())
```

Output:

```
Eksik Değerler (0'lar NaN olarak işaretlendikten sonra):
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

Description:

- **Glucose**: 5 missing values
- **BloodPressure**: 35 missing values
- **SkinThickness**: 227 missing values
- **Insulin**: 374 missing values
- **BMI**: 11 missing values

This analysis revealed a particularly high number of missing values in the **Insulin** and **SkinThickness** columns. To address this, the missing values were filled using the **median values** of the respective columns. The median was chosen to mitigate the impact of outliers, ensuring a reliable data imputation process.

After completing the filling operation, the dataset was rechecked, and it was confirmed that no missing values remained. The dataset is now ready for modeling.

```
# Eksik değerlerin medyan ile doldurulması
for column in columns_with_zeros:
    data[column].fillna(data[column].median(), inplace=True)

# Eksik değerlerin tekrar kontrolü
print("\nEksik Değerler (Doldurulduktan Sonra):")
print(data.isnull().sum())
```

Output:

```
Eksik Değerler (Doldurulduktan Sonra):
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

Description:

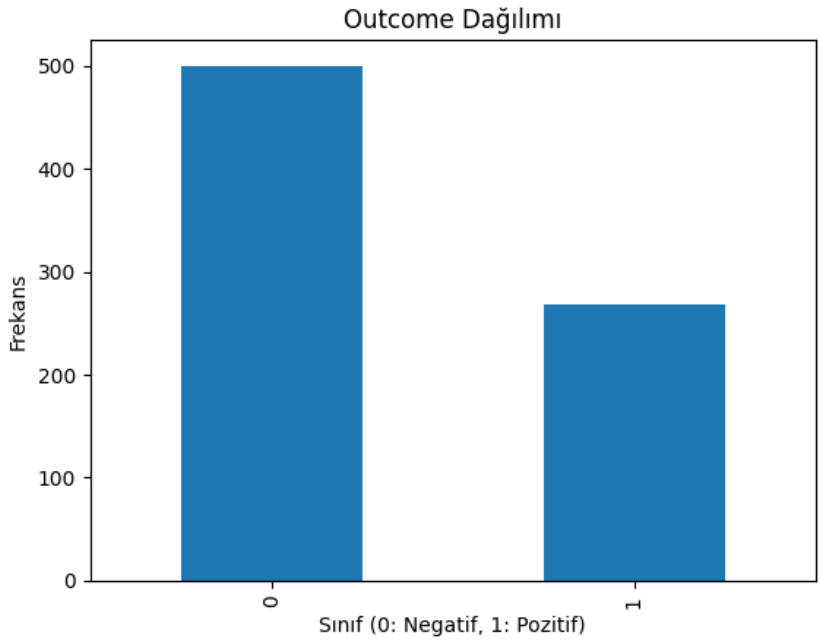
- All missing values have been successfully filled and our dataset is now missing value free.

1.2.4. Examination of the Target Variable Distribution

The class distribution of the **Outcome** column, which serves as the target variable, was analyzed to evaluate class imbalance within the dataset. The following observations were made:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Outcome değişkeninin dağılımı
sns.countplot(data=data, x='Outcome')
plt.title('Outcome Değişkeninin Sınıf Dağılımı')
plt.xlabel('Outcome (0: Diyabet Yok, 1: Diyabet Var)')
plt.ylabel('Frekans')
plt.show()
```

Output:



Açıklama:

- **Outcome = 0 (Non-diabetic):** Approximately 500 observations.
- **Outcome = 1 (Diabetic):** Approximately 268 observations.
- **Class Imbalance:** There is a noticeable class imbalance in the dataset. The number of non-diabetic individuals significantly outweighs the number of diabetic individuals

1.2.5. Examination of the Relationship Between Features and the Target Variable

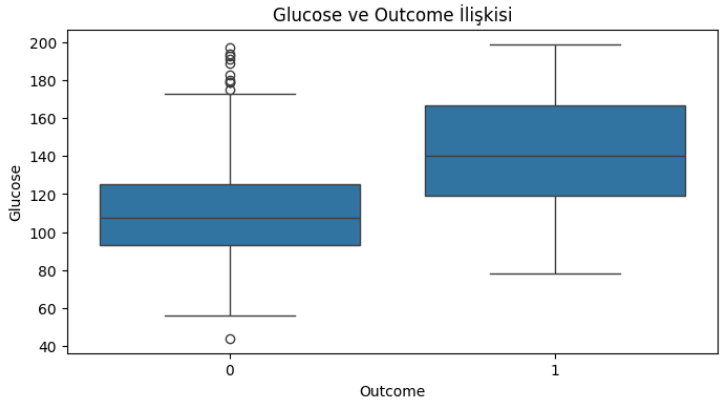
The relationships between continuous features in the dataset and the target variable (Outcome) were analyzed and visualized using boxplots. Below are the observations and analyses for each feature:

```
# Sürekli değişkenlerin Outcome ile ilişkisi
columns_to_plot = ['Glucose', 'BloodPressure', 'BMI', 'Age',
'Insulin']
for column in columns_to_plot:
    plt.figure(figsize=(8, 4))
    sns.boxplot(data=data, x='Outcome', y=column)
    plt.title(f'{column} Değerlerinin Outcome\'a Göre Dağılımı')
    plt.show()
```

Output:

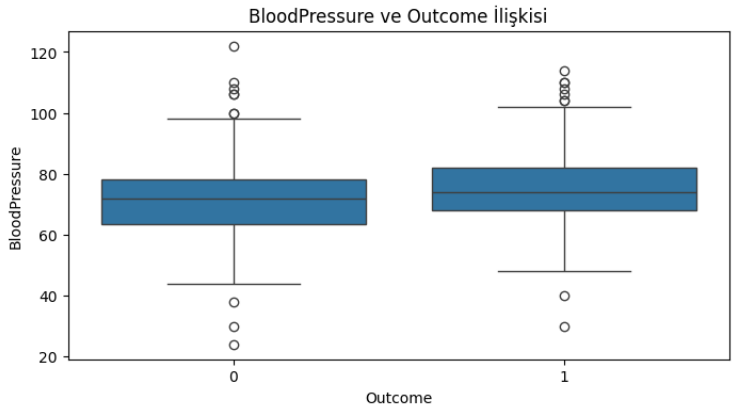
Glucose and Outcome Relationship:

- The glucose levels of diabetic individuals (**Outcome = 1**) are significantly higher compared to non-diabetic individuals (**Outcome = 0**).
- This indicates that glucose levels are a critical feature for diabetes detection.



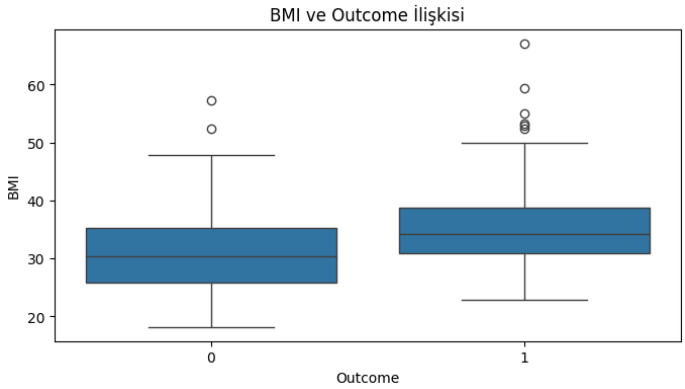
BloodPressure and Outcome Relationship:

- No significant differences were observed in blood pressure distributions between diabetic and non-diabetic individuals.
- However, some outliers in blood pressure values were identified.



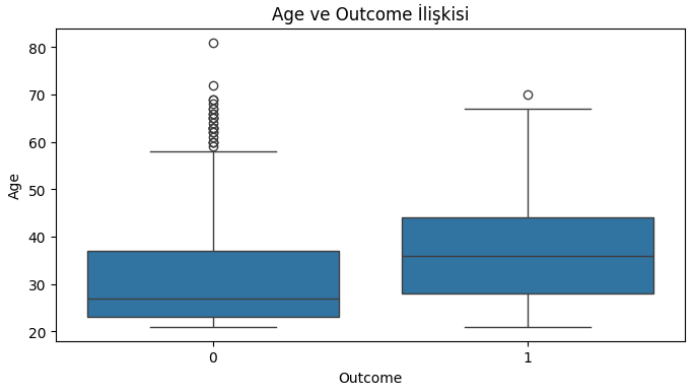
BMI and Outcome Relationship:

- Diabetic individuals (**Outcome = 1**) tend to have higher BMI (Body Mass Index) values compared to non-diabetic individuals.
- This suggests that BMI is significantly associated with diabetes.



Age and Outcome Relationship:

- The average age of diabetic individuals is higher than that of non-diabetic individuals.
- The likelihood of diabetes increases with age, highlighting age as an important factor.



Insulin and Outcome Relationship:

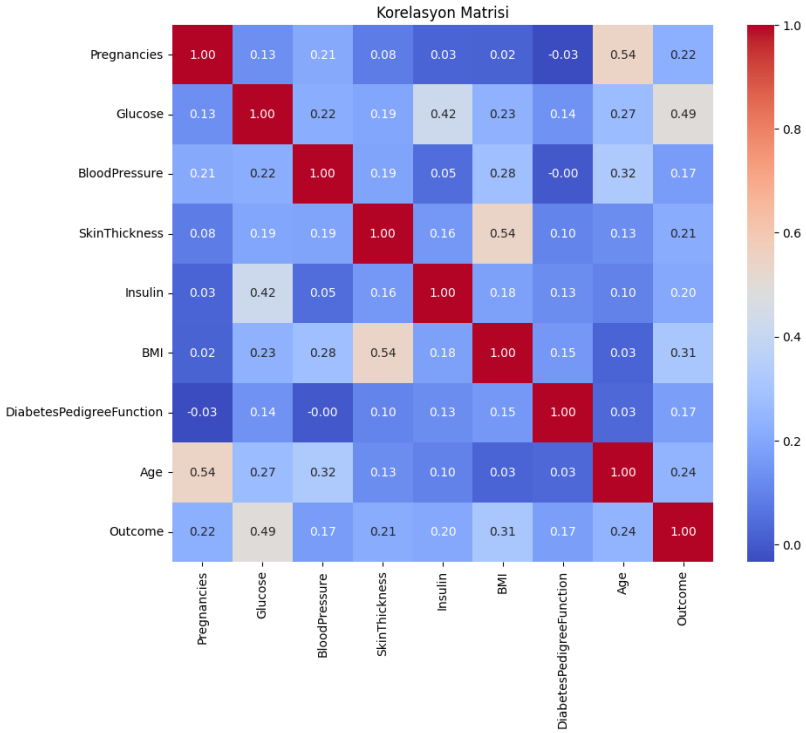
- No clear differences in insulin levels were observed between diabetic and non-diabetic individuals.
- However, the insulin data contains a considerable number of outliers.

Conclusion:
Features such as **Glucose**, **BMI**, and **Age** show a significant relationship with the target variable and should be included in the model. Other features require further analysis during data cleaning and preprocessing steps.

1.2.6. Correlation Analysis

To examine the relationships between features in the dataset, a correlation matrix was created and visualized. The following observations were made:

```
# Korelasyon matrisi ve heatmap
plt.figure(figsize=(10, 8))
corr = data.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Korelasyon Matrisi')
plt.show()
```



Output:

- **Features with the Highest Correlation to Outcome**:
  - **Glucose**: 0.49
  - **BMI**: 0.31
  - **Age**: 0.24
- There are no highly correlated features in the dataset. This indicates a low risk of **multicollinearity**, which could otherwise negatively impact the model.

Conclusion:
Glucose, BMI, and Age variables show a significant relationship with the Outcome variable and should be prioritized in the modeling process. Additionally, the low correlation between features suggests that the dataset does not suffer from multicollinearity, which supports the reliability of the model.

1.2.7. Handling Outliers

Outliers were identified in the **Insulin**, **SkinThickness**, and **BloodPressure** columns. Since these values could negatively impact data analysis and modeling processes, they were handled using the **IQR (Interquartile Range)** method.

IQR Method:

- **Q1 (1st Quartile):** Represents the first 25% of the data.
- **Q3 (3rd Quartile):** Represents the first 75% of the data.
- **IQR (Interquartile Range):** Calculated as Q3 - Q1.
- The boundaries for outliers are defined as **Lower Bound (Q1 - 1.5 * IQR)** and **Upper Bound (Q3 + 1.5 * IQR)**.

```
# IQR yöntemiyle aykırı değerlerin sınırlandırılması
for column in ['Insulin', 'SkinThickness', 'BloodPressure']:
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    data[column] = data[column].clip(lower_bound, upper_bound)

# İşlem sonrası sütunların güncel istatistikleri
print(data[['Insulin', 'SkinThickness',
'BloodPressure']].describe())
```

Output: As a result of this process, the distribution of the columns became more normalized. The updated statistics are as follows:

```
           Insulin  SkinThickness  BloodPressure
count  768.000000     768.000000     768.000000
mean   124.691081      28.866536      72.358073
std      7.913595       7.442353      11.697097
min    112.875000      14.500000      40.000000
25%    121.500000      25.000000      64.000000
50%    125.000000      29.000000      72.000000
75%    127.250000      32.000000      80.000000
max    135.875000      42.500000     104.000000
```

- **Insulin:** Mean value 124.69, standard deviation 7.91, values are constrained between 112.87 and 135.87.
- **SkinThickness:** Mean value 28.86, standard deviation 7.44, values are constrained between 14.50 and 42.50.
- **BloodPressure:** Mean value 72.36, standard deviation 11.69, values are constrained between 40.00 and 104.00.

Conclusion:
By limiting outliers, the consistency of the dataset has been improved, and the risk of negatively impacting model performance has been minimized.

1.2.8. Feature Engineering

To enhance model performance, new features were added to the dataset. The transformation applied to the age variable is detailed below.

1.2.8.1. Creating Age Groups

Definition of Age Groups:
The age variable was divided into specific intervals to create categories such as **Young**, **Middle-Aged**, **Old**, and **Very Old**. These groups were created to better analyze the impact of age on diabetes.

Categories:

- **20-30:** Young
- **30-40:** Middle-Aged
- **40-50:** Old
- **50 and above:** Very Old

```
# Yaş gruplarının oluşturulması
bins_age = [20, 30, 40, 50, 100]
labels_age = ['Genç', 'Orta Yaşlı', 'Yaşlı', 'Çok Yaşlı']
data['AgeGroup'] = pd.cut(data['Age'], bins=bins_age,
labels=labels_age)

# Yaş gruplarının dağılımı
print("\nYaş Grupları Dağılımı:")
print(data['AgeGroup'].value_counts())
```

Output:

```
Yaş Grupları Dağılımı:
Genç          417
Orta Yaşlı    157
Yaşlı         113
Çok Yaşlı      81
Name: AgeGroup, dtype: int64
```

Conclusion:
This transformation allows for a more detailed analysis of the impact of age groups on the target variable. Such feature engineering can enhance the model's learning capacity and improve results.

1.2.8.2. Creating Pregnancy Groups

Definition of Pregnancy Groups:
The number of pregnancies variable was divided into specific intervals to create categories such as **Low**, **Medium**, and **High**. These groups were designed to better analyze the impact of the number of pregnancies on diabetes.

Categories:

- **-1 to 2:** Low
- **2 to 6:** Medium
- **6 and above:** High

```
# Gebelik gruplarının oluşturulması
bins_preg = [-1, 2, 6, 20]
labels_preg = ['Düşük', 'Orta', 'Yüksek']
data['PregnanciesGroup'] = pd.cut(data['Pregnancies'],
bins=bins_preg, labels=labels_preg)

# Gebelik gruplarının dağılımı
print("\nGebelik Grupları Dağılımı:")
print(data['PregnanciesGroup'].value_counts())
```

Output:

```
Gebelik Grupları Dağılımı:
Düşük      349
Orta       250
Yüksek     169
Name: PregnanciesGroup, dtype: int64
```

Conclusion:
This transformation allows for a more detailed analysis of the impact of pregnancy groups on the target variable. These groups, along with the **AgeGroup** feature, can be included as categorical variables in the model to improve its performance.

1.2.9. Feature Scaling

To improve model performance, continuous variables in the dataset were scaled using **StandardScaler**. This process standardized the variables and eliminated issues that could arise from differing scales during the model learning process.

Process:

Scaled Columns:

- Glucose
- BloodPressure
- SkinThickness

- Insulin
- BMI
- Age

```
from sklearn.preprocessing import StandardScaler

# Ölçeklenecek sütunlar
columns_to_scale = ['Glucose', 'BloodPressure', 'SkinThickness',
'Insulin', 'BMI', 'Age']

# StandardScaler ile ölçekleme
scaler = StandardScaler()
data[columns_to_scale] =
scaler.fit_transform(data[columns_to_scale])

# Ölçeklenmiş sütunların ilk 5 satırı
print("\nÖlçeklenmiş Sütunlar (İlk 5 Satır):")
print(data[columns_to_scale].head())
```

Output:

```
Ölçeklenmiş Sütunlar (İlk 5 Satır):
    Glucose  BloodPressure  SkinThickness   Insulin       BMI       Age
0  0.866045      -0.030632       0.824667  0.039062  0.166619  1.425995
1 -1.205066      -0.543914       0.017945  0.039062 -0.852200 -0.190672
2  2.016662      -0.715008       0.017945  0.039062 -1.332500 -0.105584
3 -1.073567      -0.543914      -0.788777 -1.494110 -0.633881 -1.041549
4  0.504422      -2.768136       0.824667  1.414175  1.549303 -0.020496
```

Explanation:

- Continuous variables were standardized using **StandardScaler**.
- As a result, the features were scaled to have a **mean of 0** and a **standard deviation of 1**.
- This scaling eliminates the effects of differing scales between variables, helping the model produce more consistent results.

1.3. Data Splitting

In this step, the dataset was split into **training** and **testing** sets to prepare for the modeling process. The training set will be used for model learning, while the testing set will evaluate model performance. The splitting process was conducted to maintain the class balance of the target variable.

1.3.1. Splitting Training and Testing Sets

The dataset was split into 80% training and 20% testing sets using the **train_test_split** function. The **stratify** parameter was used to ensure that the class proportions of the target variable, **Outcome**, are maintained in both sets.

Process:

- **Independent Variables (X):**
  - The **Outcome**, **AgeGroup**, and **PregnanciesGroup** columns were excluded to focus the model solely on continuous and core independent variables.
- **Dependent Variable (y):**
  - The **Outcome** column was designated as the target variable.

```
from sklearn.model_selection import train_test_split

# Bağımsız değişkenler (X) ve bağımlı değişken (y) ayrımı
X = data.drop(['Outcome', 'AgeGroup', 'PregnanciesGroup'], axis=1)
# Outcome ve grupları çıkarıyoruz
y = data['Outcome']

# Veriyi eğitim ve test setlerine bölme
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

# Veri seti boyutlarını kontrol edelim
print("Eğitim Verisi Boyutu:", X_train.shape)
print("Test Verisi Boyutu:", X_test.shape)
```

Output:

```
Eğitim Verisi Boyutu: (614, 8)
Test Verisi Boyutu: (154, 8)
```

Explanation:

- **Training Set:** Comprises 614 observations (80% of the data).
- **Testing Set:** Comprises 154 observations (20% of the data).
- **Stratify Parameter:** Ensures that class proportions of the target variable are balanced across both sets.

Conclusion:
This step ensures that the model gains a generalizable structure during training and is fairly evaluated during testing.

1.4. Modeling and Evaluation

In this section, the **Decision Tree** model was trained and evaluated on the test set. The performance of the model was analyzed using metrics such as accuracy, precision, recall, and F1-Score. Below are the detailed steps and results for the Decision Tree model.

1.4.1. Decision Tree Model

The Decision Tree model was defined using the sklearn library and trained on the training set. Predictions were made using the test set, and performance metrics were calculated.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Karar Ağacı modelini tanımlayalım
dt_model = DecisionTreeClassifier(random_state=42)

# Modeli eğitelim
dt_model.fit(X_train, y_train)

# Test seti ile tahmin yapalım
y_pred_dt = dt_model.predict(X_test)

# Performans değerlendirme
print("\nKarar Ağacı Modeli Performansı:")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_dt))

# Confusion Matrix Görselleştirme
conf_matrix_dt = confusion_matrix(y_test, y_pred_dt)
sns.heatmap(conf_matrix_dt, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix - Karar Ağacı")
plt.xlabel("Tahmin")
plt.ylabel("Gerçek")
plt.show()
```

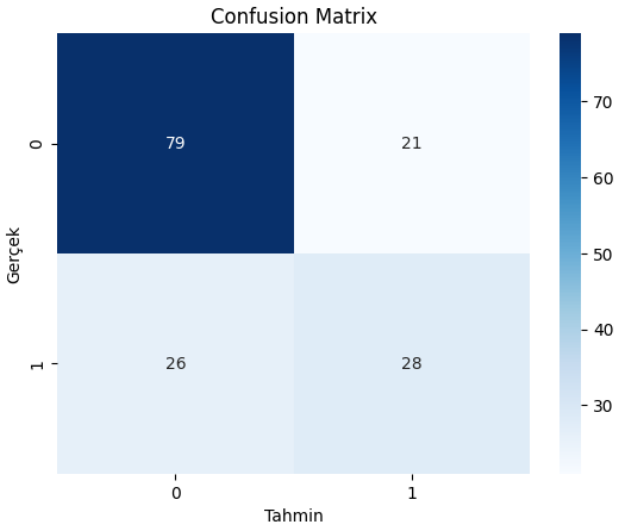
Output:

```
Karar Ağacı Modeli Performansı:
Accuracy: 0.6948051948051948

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.79      0.77       100
           1       0.57      0.52      0.54        54

    accuracy                           0.69       154
   macro avg       0.66      0.65      0.66       154
weighted avg       0.69      0.69      0.69       154
```

Performance Metrics:

- **Accuracy:** 0.69
- **Precision:**
  - Class 0 (Non-diabetic): 0.75
  - Class 1 (Diabetic): 0.57
- **Recall:**
  - Class 0 (Non-diabetic): 0.79
  - Class 1 (Diabetic): 0.52
- **F1-Score:**
  - Class 0 (Non-diabetic): 0.77
  - Class 1 (Diabetic): 0.54

Confusion Matrix:



Conclusion:

- The model performs better in correctly predicting non-diabetic individuals (79% accuracy) but struggles to identify diabetic individuals (52% accuracy).
- **F1-Score** and other metrics indicate that the overall performance of the model needs improvement.
- The model seems to be affected by class imbalance. To address this issue, sampling techniques or alternative modeling strategies can be employed.

1.4.2. Random Forest Model

In this section, the **Random Forest** model was trained and evaluated on the test set. The model's performance was analyzed using metrics such as accuracy, precision, recall, and F1-Score. Below are the detailed steps and results for the Random Forest model. The **Random Forest** model was defined using the sklearn library and trained on the training set with **100 estimators.** Predictions were made on the test set, and performance metrics were calculated.

```
from sklearn.ensemble import RandomForestClassifier

# Random Forest modelini tanımlayalım
rf_model = RandomForestClassifier(random_state=42, n_estimators=100)

# Modeli eğitelim
rf_model.fit(X_train, y_train)

# Test seti ile tahmin yapalım
y_pred_rf = rf_model.predict(X_test)

# Performans değerlendirme
print("\nRandom Forest Modeli Performansı:")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

# Confusion Matrix Görselleştirme
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(conf_matrix_rf, annot=True, fmt="d", cmap="Greens")
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Tahmin")
plt.ylabel("Gerçek")
plt.show()
```

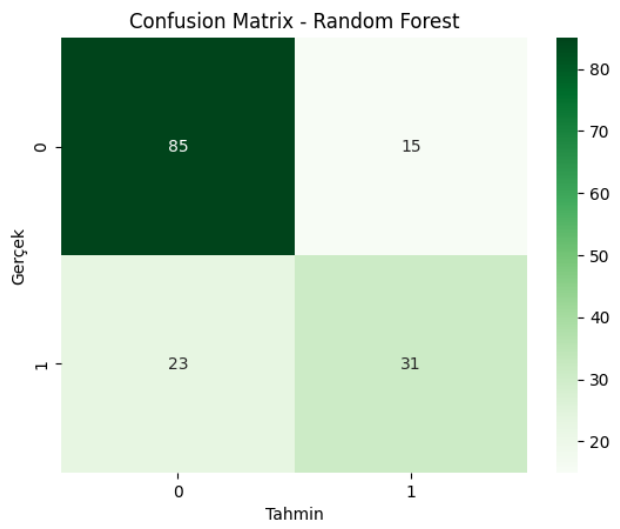
Output:

```
Random Forest Modeli Performansı:
Accuracy: 0.7532467532467533

Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.85      0.82       100
           1       0.67      0.57      0.62        54

    accuracy                           0.75       154
   macro avg       0.73      0.71      0.72       154
weighted avg       0.75      0.75      0.75       154
```

Performance Metrics:

- **Accuracy:** 0.75
- **Precision:**
  - Class 0 (Non-diabetic): 0.79
  - Class 1 (Diabetic): 0.67
- **Recall:**
  - Class 0 (Non-diabetic): 0.85
  - Class 1 (Diabetic): 0.57
- **F1-Score:**
  - Class 0 (Non-diabetic): 0.82
  - Class 1 (Diabetic): 0.62

Confusion Matrix:



Conclusion:

- The Random Forest model performs better in identifying non-diabetic individuals (85% accuracy).
- It shows limited success in detecting diabetic individuals (57% accuracy).
- The model's accuracy and other metrics are higher compared to the Decision Tree model.
- The effects of class imbalance are still noticeable. Performance can be further improved by incorporating class weights or sampling techniques.

1.4.3. Model Comparison

This section compares the performance metrics of the **Decision Tree** and **Random Forest** models. The table below summarizes their accuracy, precision, recall, and F1-Score for both Class 0 and Class 1:

Model	Accuracy	Class 0 Precision	Class 0 Recall	Class 1 Precision	Class 1 Recall	Class 1 F1-Score
Decision Tree	0.6948	0.75	0.79	0.57	0.52	0.54
Random Forest	0.7532	0.79	0.85	0.67	0.57	0.62

Analysis:

1. **Accuracy:**
   - The **Random Forest** model (75.32%) outperformed the **Decision Tree** model (69.48%) in terms of accuracy.
2. **Class 0 (Non-diabetic):**
   - **Precision:** Both models performed well, but Random Forest (0.79) was slightly better than Decision Tree (0.75).
   - **Recall:** Random Forest (85%) demonstrated better recall compared to Decision Tree (79%).
3. **Class 1 (Diabetic):**
   - **Precision:** Random Forest (67%) showed higher precision than Decision Tree (57%).
   - **Recall:** Random Forest (57%) outperformed Decision Tree (52%) in recall.
   - **F1-Score:** Random Forest (62%) achieved a higher F1-Score than Decision Tree (54%).

Conclusion:

- The **Random Forest** model outperformed the **Decision Tree** model across all metrics.
- It was particularly better at predicting diabetic individuals (Class 1).
- These results indicate that the **Random Forest** model is a more effective choice for this dataset.

1.4.4. Strengths and Weaknesses

Model	Strengths	Weaknesses
Decision Tree	- Fast training and prediction time. - Easy to interpret. - Does not create an overly complex model.	- Lower accuracy. - More likely to be affected by class imbalance. - Prone to overfitting.
Random Forest	- Higher accuracy and recall. - Reduces overfitting risk.. - Performs better on complex data.	- Higher computational cost. - Less interpretable.

1.5. Model Selection

Based on the performance results, the **Random Forest** model demonstrated better performance in both accuracy and the positive class (**Class 1 - Diabetic**).

- **Accuracy:** The Random Forest model achieved a higher accuracy rate (75.32%) compared to the Decision Tree model.
- **Positive Class Performance:** The Random Forest model outperformed the Decision Tree model in predicting diabetic individuals (Precision, Recall, and F1-Score).

Conclusion:

For this dataset and problem, the **Random Forest model** is selected as the most suitable model. Its ability to handle challenges such as class imbalance and its superior overall performance make it the preferred choice.

1.6. Conclusion

In this study, the **Diabetes Dataset** obtained from Kaggle was used to build models with the **Decision Tree** and **Random Forest** algorithms. The primary objective was to identify the most suitable model for accurately classifying diabetic patients.

Key Findings:

- The **Random Forest** model was identified as the superior model due to its higher accuracy (75.32%) and more balanced performance compared to the Decision Tree model.
- The Random Forest model demonstrated better prediction results for both negative and positive classes.

Question 2

2.1. Introduction

2.1.1. Project Objective

In this project, the **K-means algorithm**, one of the unsupervised learning methods, is used to cluster a dataset. The primary objective is to identify naturally occurring groups within the dataset, analyze these clusters, and evaluate the results. To prepare the dataset for clustering, missing values were handled, features were scaled, and the appropriate number of clusters was determined.

Dataset Used

- **Source**: **Diabetes Dataset** from Kaggle.
- **Number of Observations**: 768
- **Number of Features**: 8
- **Target Variable (Outcome)**: Indicates whether the person has diabetes or not. However, since K-means is an unsupervised method, this variable was removed from the analysis.

2.1.2. About the Dataset

In this project, the Diabetes Dataset obtained from the Kaggle platform was used. The dataset includes various health measurements of individuals and indicates whether they have diabetes or not.

- **Total Number of Observations:** 768
- **Number of Columns**: 9
- **Target Variable (Outcome)**:
    - 0: The individual is not diabetic.
    - 1: The individual is diabetic.
- **Independent Variables**:
    - **Pregnancies**: Number of pregnancies.
    - **Glucose**: Blood glucose level.
    - **BloodPressure**: Blood pressure.
    - **SkinThickness**: Skin thickness.
    - **Insulin**: Insulin level.
    - **BMI**: Body Mass Index.
    - **DiabetesPedigreeFunction**: Diabetes risk factor (genetic predisposition).
    - **Age**: Age.

2.1.3. Project Process

The project involves the following steps:

1. **Data Exploration and Preprocessing**:
    - Identifying and handling missing values in the dataset.

    - Limiting outliers.
    - Scaling continuous variables.
2. **Modeling**:
    - Applying the K-means clustering algorithm.
    - Determining the number of clusters (using Elbow Method and Silhouette Score).
3. **Performance Evaluation**:
    - Evaluating the quality of clustering using metrics such as Silhouette Score.
    - Interpreting the created clusters.
4. **Results Analysis and Interpretation**:
    - Interpreting the key characteristics of the clusters and supporting the analysis with visualizations.

2.2 Data Preparation and Preprocessing

The **data preparation and preprocessing** stage is a critical step to ensure the proper functioning of the **K-means algorithm**. In this section, missing values were handled, features were scaled, and the dataset was made ready for analysis.

2.2.1. Handling Missing Values

The issue of missing values was addressed carefully as it could impact the analysis and the performance of the K-means algorithm. In the dataset, **0 values** were considered as missing data and filled using appropriate methods.

```
# Eksik veri olarak kabul edilen 0 değerlerini NaN ile değiştirelim
import numpy as np

columns_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness',
'Insulin', 'BMI']
features[columns_with_zeros] =
features[columns_with_zeros].replace(0, np.nan)

# Eksik değerlerin sayısını kontrol edelim
print("Eksik Değerler (0'lar NaN olarak işaretlendikten sonra):")
print(features.isnull().sum())
```

Output:

```
Eksik Değerler (0'lar NaN olarak işaretlendikten sonra):
Pregnancies                   0
Glucose                       5
BloodPressure                35
SkinThickness               227
Insulin                     374
BMI                          11
DiabetesPedigreeFunction      0
Age                           0
dtype: int64
```

Description:

- **Glucose**: 5 missing values
- **BloodPressure**: 35 missing values
- **SkinThickness**: 227 missing values
- **Insulin**: 374 missing values
- **BMI**: 11 missing values

This analysis revealed a particularly high number of missing values in the **Insulin** and **SkinThickness** columns. To address this, the missing values were filled using the **median values** of the respective columns. The median was chosen to mitigate the impact of outliers, ensuring a reliable data imputation process.

After completing the filling operation, the dataset was rechecked, and it was confirmed that no missing values remained. The dataset is now ready for modeling.

```
# Eksik değerleri medyan ile dolduralım
for column in columns_with_zeros:
    features[column].fillna(features[column].median())

# Eksik değerlerin tekrar kontrolü
print("\nEksik Değerler (Doldurulduktan Sonra):")
print(features.isnull().sum())
```

Output:

```
Missing Values (After Filling):
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
dtype: int64
```

Description:

- All missing values have been successfully filled and our dataset is now missing value free.
- **Missing Value Handling:** Missing values were filled using the median of each respective column.
- **Result:** The missing value issue has been successfully resolved, and no missing values remain in the dataset. This ensures that the **K-means algorithm** can operate effectively without being impacted by missing data.

2.3. Feature Scaling

K-means algorithm is a distance-based method, so all features need to be on the same scale. In this step, all features in the dataset were standardized using **StandardScaler**. Standardization transformed each feature to have a mean of **0** and a standard deviation of **1**.

```
from sklearn.preprocessing import StandardScaler

# Ölçekleme işlemi
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)

# Ölçeklenmiş özelliklerin DataFrame olarak görüntülenmesi
scaled_features = pd.DataFrame(scaled_features,
columns=features.columns)

print("\nÖlçeklenmiş Özellikler (İlk 5 Satır):")
print(scaled_features.head())
```

Description:

- Using **StandardScaler**, all features were transformed to the same scale (mean **0**, standard deviation **1**).
- This ensures that the **K-means algorithm** produces accurate and effective results in its distance-based calculations.
- This process resolves any imbalances caused by differing feature scales.

2.4. Determining the Optimal Number of Clusters with the Elbow Method

In the K-means algorithm, selecting the correct number of clusters (**k**) is a critical step. The choice of the number of clusters directly affects the performance and accuracy of the model. In this step, the **Elbow Method** was used to determine the optimal number of clusters.

What is the Elbow Method?

The Elbow Method involves calculating the total intra-cluster variation (inertia) for each k value and visualizing it on a graph.

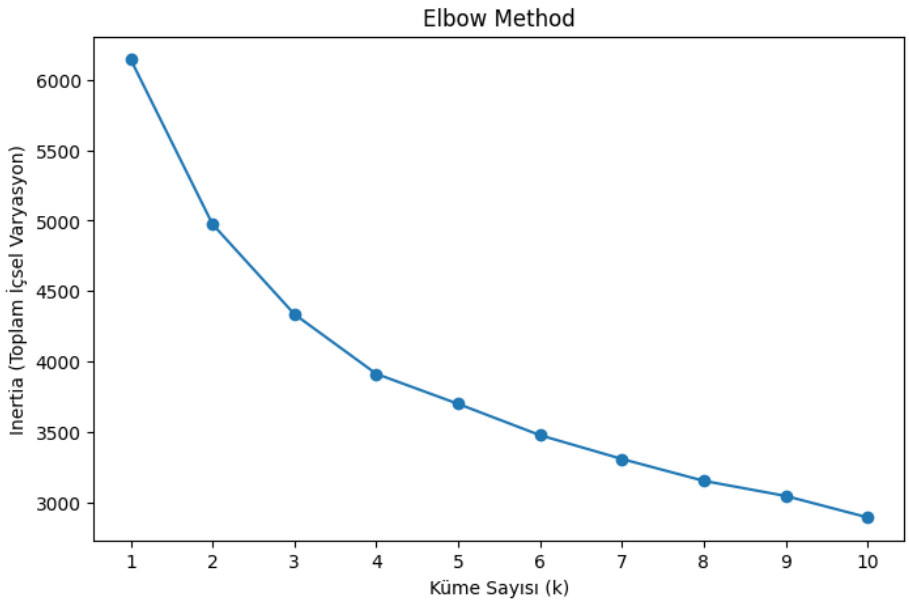
- **Inertia:** It is the sum of squared distances between points and their respective cluster centers.
- **Objective:** To identify the "elbow" point on the graph, where the variation between clusters decreases significantly, and further increasing the k value yields diminishing returns.

Implementation Steps

1. **Calculating Inertia Values:**
   - Inertia values were calculated for different cluster counts (**k**) ranging from 1 to 10.
   - A K-means model was run for each k value, and the inertia values were stored in a list.
2. **Plotting the Graph:**
   - The inertia values were visualized using a line graph against the k values.

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Farklı k değerleri için inertia hesaplama
inertia = []
k_values = range(1, 11)
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(scaled_features)
    inertia.append(kmeans.inertia_)

# Elbow Method grafiği
plt.figure(figsize=(8, 5))
plt.plot(k_values, inertia, '-o')
plt.title('Elbow Method')
plt.xlabel('Küme Sayısı (k)')
plt.ylabel('Inertia (Toplam İçsel Varyasyon)')
plt.xticks(k_values)
plt.show()
```

Output:



Analysis of the Graph

The following observations can be made from the graph:

- **Objective:** To identify the "elbow" point, where the slope of the curve significantly decreases.
- **Observation:**
  - The graph shows a noticeable elbow at k = 3 and k = 4.
  - These points suggest that the optimal number of clusters is likely 3 or 4.

2.4.1. Validating Cluster Count with Silhouette Score

Why Use Silhouette Score?

While the **Elbow Method** provides a good starting point for determining the optimal number of clusters, it is not sufficient for definitive conclusions. To gain deeper insights into the quality of the clusters, metrics such as **Silhouette Score** should be used.

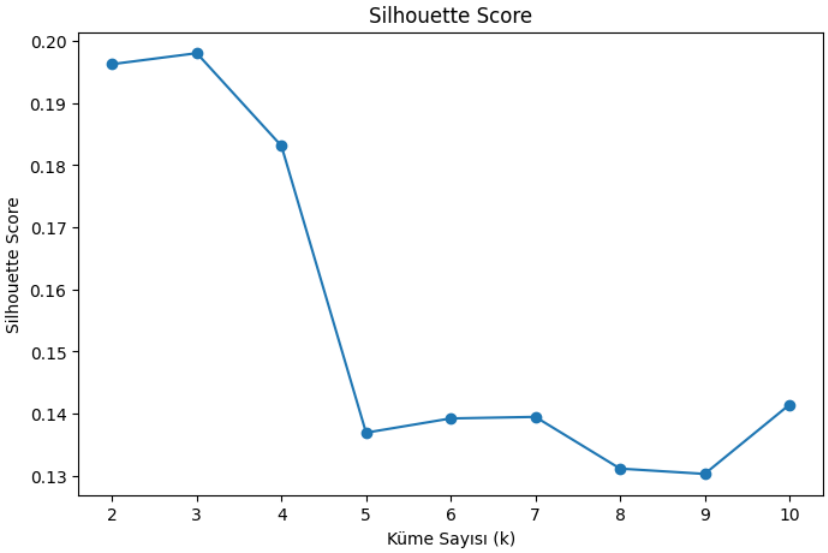
What is Silhouette Score?

The **Silhouette Score** measures how well-separated the clusters are and their internal cohesion. The score is interpreted as follows:

- **Close to 1:** Clusters are well-separated and internally cohesive.
- **0:** Clusters overlap significantly.
- **Negative:** Clusters are misassigned.

```
from sklearn.metrics import silhouette_score

# Farklı k değerleri için Silhouette Score hesaplama
silhouette_scores = []
k_values = range(2, 11)  # 2'den başlayarak hesaplama yapılır
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(scaled_features)
    score = silhouette_score(scaled_features, kmeans.labels_)
    silhouette_scores.append(score)

# Silhouette Score grafiği
plt.figure(figsize=(8, 5))
plt.plot(k_values, silhouette_scores, '-o')
plt.title('Silhouette Score')
plt.xlabel('Küme Sayısı (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_values)
plt.show()
```

Output:



Analysis of the Silhouette Score Graph

The **Silhouette Score** graph was analyzed as follows:

- **Objective**: Identify the k value with the highest Silhouette Score, as this indicates the best-separated clusters.
- **Observations**:
  - The highest Silhouette Score was observed at **k = 2** (approximately 0.20).
  - For **k = 3** and **k = 4**, the scores were close, but **k = 3** performed slightly better than **k = 4**.
  - For higher k values (e.g., $k \geq 5$), the Silhouette Score dropped significantly, indicating poor cluster separation.

Conclusion:

- **Optimal Number of Clusters:**
  - Based on both the **Elbow Method** and **Silhouette Score** analysis, **k = 3** is the most suitable number of clusters.
  - This result indicates that the clusters are well-separated and have good internal cohesion.
- **Recommendations:**
  - **Model Performance:** Selecting **k = 3** provides a solid foundation for improving model performance.
  - **Further Evaluation:** If necessary, other validation metrics (e.g., Dunn Index) can be applied for additional insights.

2.5 K-means Clustering and PCA Visualization

In this step, the dataset was clustered into groups using the K-means algorithm. To visualize the distribution and separation quality of the clusters, dimensionality reduction was performed using PCA (Principal Component Analysis).

K-means Clustering:

- The K-means algorithm was applied with **k = 3** clusters.
- Each observation was assigned to its respective cluster, and this information was saved in a new column called **Cluster**.

```
# K-means algoritmasını uygulama
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(scaled_features)

# Kümeleri ekleyelim
scaled_features['Cluster'] = kmeans.labels_

# Küme merkezlerini görüntüleyelim
print("Küme Merkezleri:")
print(kmeans.cluster_centers_)

# Her kümedeki veri sayısını kontrol edelim
print("\nKüme Dağılımları:")
print(scaled_features['Cluster'].value_counts())
```

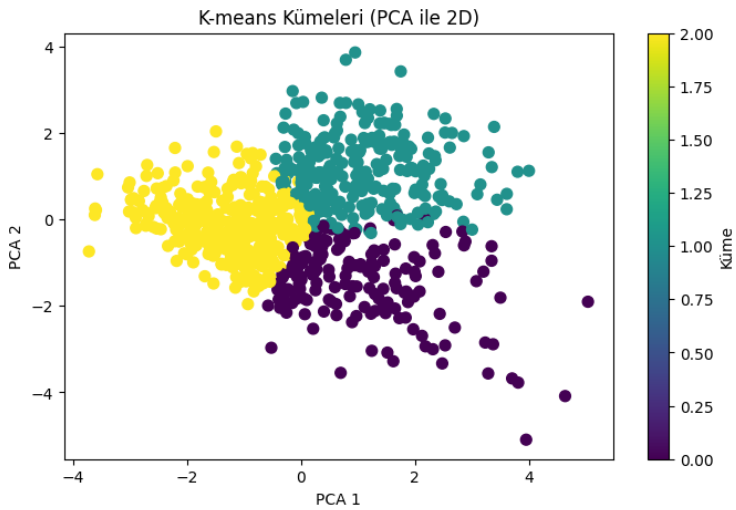
Cluster Centers:

- The K-means algorithm identified the following cluster centers:

```
Küme Merkezleri:
[[-0.56189348  0.64829774  0.15534981  0.83535251  0.75648111
0.98518491
   0.44310522 -0.3328185 ]
 [ 1.02460067  0.26368576  0.50374541  0.13513212 -0.05375363
0.06330493
  -0.0522288   1.05415522]
 [-0.47885662 -0.5150779  -0.44872326 -0.51259734 -0.33415223 -
0.53360723
  -0.18041213 -0.61388454]]

Küme Dağılımları:
Cluster
2    344
1    254
0    170
Name: count, dtype: int64
```

- These centers represent the average feature values for each cluster.

PCA Visualization:



1. **PCA Dimensionality Reduction:**
   - The original dataset was reduced from 8 dimensions to 2 dimensions. PCA selected the two components that explain the most variance in the dataset.
2. **Visualizing the Clusters:**
   - The clusters were visualized on a 2D scatter plot using the PCA-reduced data.
   - Different colors represent the clusters:
     - **Cluster 0:** Yellow
     - **Cluster 1:** Green
     - **Cluster 2:** Purple
   - The plot shows that the clusters are well-separated, with some minor overlaps in certain regions.

Conclusion and Remarks:

- **Clustering Performance:** The K-means algorithm successfully grouped the data into three clusters.
- **PCA Visualization:** PCA revealed clear separation between clusters in a two-dimensional space, though slight overlaps exist in some areas.
- **Distribution:** Cluster 2 was observed to be the largest group, indicating that a significant portion of the data belongs to this cluster.

2.6 K-means Clustering Results and Cluster Analysis

In this study, the dataset was divided into three clusters with the K-means algorithm and the values of the features in the original scales were analysed for each cluster. Cluster analyses provide a detailed insight into the demographic and physical characteristics of groups in terms of diabetes risk.

```
# Küme merkezlerini orijinal ölçeğe döndürelim
original_centers = scaler.inverse_transform(kmeans.cluster_centers_)

# Orijinal küme merkezlerini bir DataFrame olarak görüntüleyelim
cluster_centers_df = pd.DataFrame(original_centers,
columns=features.columns)
print("Orijinal Küme Merkezleri:")
print(cluster_centers_df)

# Her kümenin özelliklerini ayrı ayrı inceleyelim
for i, center in cluster_centers_df.iterrows():
    print(f"\nKüme {i}:")
    print(center)
```

Output:

Cluster Centers (Original Scale)

Below are the average values (cluster centers) for each cluster:

Feature	Cluster 0	Cluster 1	Cluster 2
Pregnancies	1.95	7.30	2.23
Glucose	141.37	129.67	105.99
BloodPressure	74.26	78.48	66.96
SkinThickness	36.44	30.30	24.60
Insulin	205.98	136.03	111.82
BMI	39.22	32.89	28.79
DiabetesPedigreeFunction	0.62	0.45	0.41
Age	29.33	45.63	26.03

Cluster Analysis

Cluster 0:

- **Characteristics:**
  - **Low pregnancy count** (average 1.95).
  - **High glucose level** (average 141.37).
  - **Average blood pressure** (average 74.26).
  - **High skin thickness** (average 36.44).
  - **High insulin level** (average 205.98).
  - **High BMI** (average 39.22).
  - **High genetic risk** (average 0.62).
  - **Young individuals** (average age 29.33).
- This cluster comprises young individuals but exhibits high glucose, insulin, BMI, and skin thickness. These features suggest a high risk of diabetes for this group.

Cluster 1:

- **Characteristics:**
  - **High pregnancy count** (average 7.30).
  - **High glucose level** (average 129.67).
  - **Slightly elevated blood pressure** (average 78.48).
  - **Average skin thickness** (average 30.30).
  - **Moderate insulin level** (average 136.03).
  - **High BMI** (average 32.89).
  - **Moderate genetic risk** (average 0.45).
  - **Middle-aged individuals** (average age 45.63).
- This cluster represents older individuals with high pregnancy counts and BMI, suggesting a significant risk of diabetes.

Cluster 2:

- **Characteristics:**
  - **Low pregnancy count** (average 2.23).
  - **Lower-than-normal glucose level** (average 105.99).
  - **Low blood pressure** (average 66.96).
  - **Low skin thickness** (average 24.60).
  - **Normal insulin level** (average 111.82).
  - **Moderate BMI** (average 28.79).
  - **Low genetic risk** (average 0.41).
  - **Young individuals** (average age 26.03).
- This cluster represents young individuals with lower glucose, BMI, and genetic risk, indicating a low risk of diabetes.

Overall Evaluation

1. **Cluster 0:**
   - Represents young individuals.
   - High glucose, insulin, and BMI levels indicate a high risk of diabetes.
2. **Cluster 1:**
   - Comprises middle-aged individuals.
   - High pregnancy counts and BMI suggest significant diabetes risk for this group.
3. **Cluster 2:**
   - Consists of young, low-risk individuals.
   - Exhibits low levels of glucose, blood pressure, and genetic risk.

Final Summary

In this study, **K-means clustering** was applied to group individuals into distinct clusters based on key health indicators such as glucose level, BMI, insulin, and age. The analysis aimed to identify patterns and potential diabetes risk within the dataset.

Key Findings

1. **Cluster 0:**
   - Young individuals with **high glucose**, **insulin**, and **BMI** levels.
   - This group shows a **high risk of diabetes** despite their younger age.
2. **Cluster 1:**
   - Middle-aged individuals with **high pregnancy counts**, **BMI**, and moderately high glucose levels.
   - This group is also at **significant diabetes risk**, likely due to their physical and demographic characteristics.
3. **Cluster 2:**
   - Young individuals with **low glucose**, **BMI**, and **genetic risk factors**.
   - This group represents a **low-risk category** for diabetes.

Methodology

1. **Feature Scaling:**
   - All features were standardized to ensure consistency in distance-based clustering.
2. **K-means Algorithm:**
   - Based on the **Elbow Method** and **Silhouette Score**, the optimal number of clusters was determined as **k = 3**.
   - Clusters were analyzed in both standardized and original scales.
3. **Visualization with PCA:**
   - PCA was employed to reduce dimensionality and visualize cluster separations in 2D space, confirming meaningful grouping with minor overlaps.
4. **Cluster Center Analysis:**
   - Cluster centers were analyzed on the original scale, providing detailed insights into the average health characteristics of each group.

Conclusion

This clustering analysis successfully segmented individuals into three distinct groups based on health metrics, providing insights into diabetes risk:

- **High-risk groups (Clusters 0 and 1):** These groups should receive targeted interventions and further monitoring.
- **Low-risk group (Cluster 2):** While this group poses minimal risk, regular health monitoring is still recommended to maintain their status.

This study demonstrates the effectiveness of **K-means clustering** combined with **PCA visualization** for identifying meaningful patterns and potential health risks within datasets.

Question 3

3.1. Introduction

3.1.1. Project Objective

In this project, **Regression** techniques, a supervised learning approach, were employed to predict a continuous target variable in a dataset. The main objective was to evaluate the performance of regression models and observe the impact of advanced regularized models on predictive accuracy.

Dataset Used

- **Source**: **Diabetes Dataset** from Kaggle.
- **Number of Observations**: 768
- **Number of Features**: 8
- **Target Variable (Outcome)**: Indicates whether the person has diabetes or not. However, since K-means is an unsupervised method, this variable was removed from the analysis.

3.1.2. About the Dataset

In this project, the Diabetes Dataset obtained from the Kaggle platform was used. The dataset includes various health measurements of individuals and indicates whether they have diabetes or not.

- **Total Number of Observations:** 768
- **Number of Columns**: 9
- **Target Variable (Outcome)**:
    - 0: The individual is not diabetic.
    - 1: The individual is diabetic.
- **Independent Variables**:
    - **Pregnancies**: Number of pregnancies.
    - **Glucose**: Blood glucose level.
    - **BloodPressure**: Blood pressure.
    - **SkinThickness**: Skin thickness.
    - **Insulin**: Insulin level.
    - **BMI**: Body Mass Index.
    - **DiabetesPedigreeFunction**: Diabetes risk factor (genetic predisposition).
    - **Age**: Age.

3.1.3. Scope of the Study

- A **Linear Regression** model was implemented, followed by a comparison with the more advanced **Ridge Regression** technique.
- The models were evaluated and compared using accuracy and error metrics.

3.2. Data Preparation and Preprocessing

In this section, the data set was made suitable for the estimation models. Filling missing values, scaling of features and separation of independent/dependent variables were performed.

The issue of missing values was addressed carefully as it could impact the analysis and the performance of the algorithm. In the dataset, **0 values** were considered as missing data and filled using appropriate methods.

```
# Eksik veri olarak kabul edilen 0 değerlerini NaN ile değiştirelim
import numpy as np

columns_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness',
'Insulin', 'BMI']
features[columns_with_zeros] =
features[columns_with_zeros].replace(0, np.nan)

# Eksik değerlerin sayısını kontrol edelim
print("Eksik Değerler (0'lar NaN olarak işaretlendikten sonra):")
print(features.isnull().sum())
```

Output:

```
Eksik Değerler (0'lar NaN olarak işaretlendikten sonra):
Pregnancies                   0
Glucose                       5
BloodPressure                35
SkinThickness               227
Insulin                     374
BMI                          11
DiabetesPedigreeFunction      0
Age                           0
dtype: int64
```

Description:

- **Glucose**: 5 missing values
- **BloodPressure**: 35 missing values
- **SkinThickness**: 227 missing values
- **Insulin**: 374 missing values
- **BMI**: 11 missing values

This analysis revealed a particularly high number of missing values in the **Insulin** and **SkinThickness** columns. To address this, the missing values were filled using the **median values** of the respective columns. The median was chosen to mitigate the impact of outliers, ensuring a reliable data imputation process.

After completing the filling operation, the dataset was rechecked, and it was confirmed that no missing values remained. The dataset is now ready for modeling.

```
# Eksik değerleri medyan ile dolduralım
for column in columns_with_zeros:
    features[column].fillna(features[column].median())

# Eksik değerlerin tekrar kontrolü
print("\nEksik Değerler (Doldurulduktan Sonra):")
print(features.isnull().sum())
```

Output:

```
Missing Values (After Filling):
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
dtype: int64
```

Description:

- All missing values have been successfully filled and our dataset is now missing value free.
- **Missing Value Handling:** Missing values were filled using the median of each respective column.
- **Result:** The missing value issue has been successfully resolved, and no missing values remain in the dataset. This ensures that the can operate effectively without being impacted by missing data.

3.2.1. Feature Scaling

As with K-means, scaling features is essential for regression models that rely on distance-based calculations. Here, all independent variables are standardized using **StandardScaler**. This process normalizes the features such that their mean is **0** and standard deviation is **1**.

```
from sklearn.preprocessing import StandardScaler

# Bağımsız değişkenlerin (X) ölçeklendirilmesi
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Ölçeklenmiş veri setini DataFrame olarak yeniden düzenleme
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

# Ölçeklenmiş özelliklerin ilk birkaç satırını kontrol edelim
```

```
print("\nÖlçeklendirilmiş Özellikler (İlk 5 Satır):")
print(X_scaled.head())
```

Output: As a result of the scaling process, each feature was normalised.

Comments:

- All independent variables have been scaled to the same range.
- This ensures that all variables contribute equally to the regression models.

3.3 Modeling and Evaluation

In this step, the dataset is split into training (80%) and test (20%) sets. The **Linear Regression** model is trained on the training data and predictions are made on the test data.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

# Linear Regression modeli oluşturma ve eğitme
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Test seti ile tahmin yapma
y_pred_lr = lr_model.predict(X_test)

# Performans değerlendirme
mae = mean_absolute_error(y_test, y_pred_lr)
mse = mean_squared_error(y_test, y_pred_lr)
r2 = r2_score(y_test, y_pred_lr)

print("Linear Regression Modeli Performansı:")
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("R-squared (R²):", r2)
```

Output: The Linear Regression model showed the following performance on the test set:

- **Mean Squared Error (MSE):** The mean of the squared differences between predicted and actual values.
- **Mean Squared Error (MSE):** The mean of the squared differences between predicted and actual values.
- **R^2 Score:** A metric indicating how well the model explains the data.

```
Linear Regression Modeli Performansı:
Mean Absolute Error (MAE): 19.818619496487507
Mean Squared Error (MSE): 626.8445016882793
R-squared (R²): 0.3770854597374351
```

Comments:

- The model demonstrates moderate performance on the test data.
- **R^2 Score:** Indicates that approximately 37.7% of the variance in the target variable is explained by the independent variables.
- **Mean Squared Error (MSE):** The average squared difference between predicted and actual values is 626.84.
- **Mean Absolute Error (MAE):** An average prediction error of 19.82 units.

3.3.1. Training the Ridge Regression Model

In this step, **Ridge Regression** is applied as an alternative to the Linear Regression model. Ridge Regression adds regularization to reduce the risk of overfitting. The L2 regularization term restricts the magnitude of the coefficients, enabling better generalization of the model.

```
from sklearn.linear_model import Ridge

# Ridge Regression modeli oluşturma ve eğitme
ridge_model = Ridge(alpha=1.0)  # Alpha düzenlileştirme katsayısı
ridge_model.fit(X_train, y_train)

# Test seti ile tahmin yapma
y_pred_ridge = ridge_model.predict(X_test)

# Performans değerlendirme
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print("\nRidge Regression Modeli Performansı:")
print("Mean Absolute Error (MAE):", mae_ridge)
print("Mean Squared Error (MSE):", mse_ridge)
print("R-squared (R²):", r2_ridge)
```

Output:

```
Ridge Regression Modeli Performansı:
Mean Absolute Error (MAE): 19.820792834707696
Mean Squared Error (MSE): 626.8679853783567
R-squared (R²): 0.3770621232768474
```

Comments:

- The performance of the Ridge Regression model is nearly identical to the Linear Regression model. However, the regularization effect ensures that Ridge Regression provides a more generalizable model.
- **R^2 Score:** At 37.7%, it is on par with the Linear Regression model.
- **MSE and MAE values:** Indicate that Ridge Regression maintains the same level of accuracy as Linear Regression.

3.4. Comparison and Interpretation of Results

3.4.1. Performance Comparison

Model	Mean Absolute Error (MAE)	Mean Squared Error (MSE)	R^2 Score
Linear Regression	19.82	626.84	0.3771
Ridge Regression	19.82	626.87	0.3771

1. **Comparison of Linear Regression and Ridge Regression:**
    - **MAE and MSE:** Both models show almost identical performance.
    - **R^2 Score:** Both models explain 37.7% of the variance in the target variable, indicating moderate explanatory power.
    - **Regularization Effect:** Ridge Regression, through L2 regularization, constrains the magnitude of the coefficients. This implies that Ridge Regression offers a more generalizable model, especially if the dataset becomes more complex.
2. **Overall Assessment:**
    - For this dataset, Ridge Regression does not significantly outperform Linear Regression. However, the inclusion of regularization demonstrates that Ridge Regression reduces the risk of overfitting.
    - If the dataset had more features or greater complexity, the superiority of Ridge Regression might become more apparent.

Results: This study focused on predicting the continuous target variable in the **Diabetes Dataset** using **Linear Regression** and **Ridge Regression** models. The performance of the models on the test data indicates that the **R^2 score** reflects a limited accuracy, with both models explaining 37.7% of the variance in the target variable. **MAE** and **MSE** metrics show that both models have similar levels of accuracy.